

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) №8 «Компьютерные науки и прикладная математика»
Кафедра 806
Группа М8О-403Б-22
Направление подготовки 01.03.02 Прикладная математика и информатика
Профиль Информатика
Квалификация бакалавр

УТВЕРЖДАЮ

Заведующий кафедрой №806 _____

С. С. Крылов

(№ каф.)

(подпись)

(инициалы, фамилия)

_____ 2026 г.

ЗАДАНИЕ

на выпускную квалификационную работу
бакалавра

Обучающийся Мудров Павел Федорович _____

(фамилия, имя, отчество полностью)

Руководитель Ляпина Светлана Юрьевна _____

(фамилия, имя, отчество полностью)

профессор _____

ученая степень, ученое звание, должность и место работы

1. Наименование темы «Применение методов обучения с подкреплением (Reinforcement Learning, RL) для прогнозирования и оптимизации биржевых котировок»

2. Срок сдачи обучающимся законченной работы 15.05.2026

3. Задание и исходные данные к работе

Разработать единый экспериментальный стенд сравнения baseline-стратегий, статистических подходов, моделей машинного обучения и агентов обучения с подкреплением на биржевых котировках. Реализовать программный комплекс на Python с загрузкой данных, формированием признаков, обучением моделей, расчетом метрик и визуализацией результатов.

Перечень иллюстративно-графических материалов *при наличии:

№ п/п	Наименование	Количество листов

4. Перечень подлежащих разработке разделов и этапы выполнения работы

№ п/п	Наименование раздела или этапа	Трудоемкость в % от полной трудоемкости квалификационной работы	Срок выполнения	Примечание
1	Анализ предметной области, аналогов и постановка исследовательской проблемы	15	15.02.2026	Выполнено на основе анализа литературы
2	Проектирование экспериментального стенда и постановка задач ML и RL	20	01.03.2026	Согласовано с руководителем
3	Реализация загрузки данных, генерации признаков, baseline-стратегий и ML-моделей	25	20.03.2026	Реализовано на языке Python
4	Реализация RL-среды, обучение DQN и PPO, экспериментальное моделирование	25	10.04.2026	Проведено тестирование
5	Подготовка выводов, оформление текста ВКР, рисунков и приложений	15	15.05.2026	Итоговое оформление работы

5. Исходные материалы и пособия

1. Sutton R. S., Barto A. G. *Reinforcement Learning: An Introduction*.
2. Hyndman R. J., Athanasopoulos G. *Forecasting: Principles and Practice*.
3. Mnih V. et al. *Human-level Control through Deep Reinforcement Learning*.
4. Schulman J. et al. *Proximal Policy Optimization Algorithms*.
5. Документация *stable-baselines3*, *Gymnasium*, *pandas*, *scikit-learn*, *yfinance*.

6. Дата выдачи задания 11.02.2026

Руководитель _____

(подпись)

Задание принял к исполнению

(подпись)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) №8 «Компьютерные науки и прикладная математика»
Кафедра 806
Группа М8О-403Б-22
Направление подготовки 01.03.02 Прикладная математика и информатика
Профиль Информатика
Квалификация бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
БАКАЛАВРА

На тему: «Применение методов обучения с подкреплением (Reinforcement Learning, RL) для прогнозирования и оптимизации биржевых котировок»

Автор ВКРБ _____ Мудров Павел Федорович
(_____) (фамилия, имя, отчество полностью)

Руководитель _____ Ляпина Светлана Юрьевна
(_____) (фамилия, имя, отчество полностью)

Консультант _____
(_____) (фамилия, имя, отчество полностью)

Консультант _____
(_____) (фамилия, имя, отчество полностью)

Рецензент _____
(_____) (фамилия, имя, отчество полностью)

К защите допустить

Заведующий кафедрой № 806 Крылов Сергей Сергеевич (_____) (№ каф.) (фамилия, имя, отчество полностью)

_____ 2026 г.

Москва 2026

СПИСОК ИСПОЛНИТЕЛЕЙ

В выполнении выпускной квалификационной работы принимали участие:

Обучающийся — Мудров Павел Федорович

Руководитель — Ляпина Светлана Юрьевна, профессор

Консультант — TODO: заполнить сведения при наличии

Рецензент — TODO: заполнить сведения при наличии

РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 57 страниц, 28 использованных источников, 7 таблиц, 5 рисунков и одного приложения.

ОБУЧЕНИЕ С ПОДКРЕПЛЕНИЕМ, БИРЖЕВЫЕ КОТИРОВКИ, АЛГОРИТМИЧЕСКАЯ ТОРГОВЛЯ, DQN, PPO, BASELINE-СТРАТЕГИИ, МАШИННОЕ ОБУЧЕНИЕ, ФИНАНСОВЫЕ МЕТРИКИ, EQUITY CURVE, MAX DRAWDOWN

Работа посвящена исследованию применения методов обучения с подкреплением для прогнозирования и оптимизации торговых решений на биржевых котировках. В отличие от подходов, ориентированных исключительно на прогноз будущей цены, в работе рассматривается задача последовательного выбора действий *buy*, *sell* и *hold* с учетом состояния рынка и динамики портфеля.

В теоретической части рассмотрены особенности финансовых временных рядов, классические статистические методы, модели машинного обучения, нейросетевые подходы и современные RL-алгоритмы. Особое внимание уделено сравнению аналогов и формулированию проблемы, состоящей в том, что высокая точность прогноза не гарантирует практической прибыльности стратегии. На этой основе обоснована необходимость построения единого экспериментального стенда, в котором baseline-стратегии, ML-модели и RL-агенты оцениваются на одних и тех же данных и по единому набору метрик.

В практической части описан программный проект на Python, включающий загрузку данных через *yfinance*, генерацию признаков на основе OHLCV-рядов, реализацию стратегий *Buy & Hold* и *Moving Average Crossover*, обучение моделей *Logistic Regression*, *Random Forest*, *Gradient Boosting*, а также обучение агентов *DQN* и *PPO* в среде *Gymnasium*. Для оценки результатов используются как ML-метрики, так и финансовые показатели: доходность, волатильность, коэффициент Шарпа, максимальная просадка и устойчивость стратегии.

Результаты экспериментального моделирования показывают, что RL-подходы целесообразно рассматривать как инструмент оптимизации последовательных торговых решений, однако их качество существенно зависит

от функции вознаграждения, набора признаков и рыночного режима. В условиях исследуемого стенда РРО-агент обеспечивает более устойчивый риск-скорректированный результат по сравнению с рядом альтернатив, тогда как стратегия *Buy & Hold* сохраняет преимущество по абсолютной доходности на выраженном восходящем рынке.

Содержание

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

ВВЕДЕНИЕ	1
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ АНАЛОГОВ	5
1.1 Особенности биржевых котировок как временных рядов . . .	5
1.2 Классические методы прогнозирования финансовых временных рядов	6
1.3 Методы машинного обучения для прогнозирования котировок	6
1.4 Нейросетевые модели для анализа временных рядов	7
1.5 Обучение с подкреплением в задачах алгоритмической торговли	8
1.6 Обзор аналогов и существующих решений: FinRL, TensorTrade, gym-anytrading	9
1.7 Сравнительная таблица аналогов	10
1.8 Постановка проблемы и выводы по главе	13
2 ПРОЕКТИРОВАНИЕ МЕТОДА И ЭКСПЕРИМЕНТАЛЬНОГО СТЕНДА	15
2.1 Общая архитектура исследования	15
2.2 Источники данных и структура OHLCV-данных	16
2.3 Предобработка данных	17
2.4 Формирование признаков: доходности, скользящие средние, RSI, MACD, Bollinger Bands, волатильность	18
2.5 Формирование целевых переменных для ML-моделей	19
2.6 Baseline-стратегии: Buy & Hold и Moving Average Crossover .	19
2.7 Постановка задачи обучения с подкреплением	20
2.8 Описание состояния, действий и функции вознаграждения RL-агента	21
2.9 Метрики оценки качества моделей и стратегий	22
2.10 Выводы по главе	25

3	РЕАЛИЗАЦИЯ ПРОГРАММНОГО РЕШЕНИЯ	26
3.1	Выбор инструментов разработки	26
3.2	Структура программного проекта	26
3.3	Модуль загрузки и подготовки данных	28
3.4	Модуль генерации признаков	28
3.5	Реализация baseline-стратегий	29
3.6	Реализация ML-моделей	29
3.7	Реализация торговой RL-среды	30
3.8	Обучение RL-агентов DQN и PPO	30
3.9	Модуль расчета финансовых метрик	31
3.10	Модуль визуализации результатов	31
3.11	Выводы по главе	32
4	ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ	33
4.1	Описание экспериментальных условий	33
4.2	Используемые биржевые инструменты	34
4.3	Сравнение baseline-стратегий	35
4.4	Результаты ML-моделей	35
4.5	Результаты RL-агентов	36
4.6	Сравнение всех подходов по финансовым метрикам	37
4.7	Анализ доходности	38
4.8	Анализ риска и максимальной просадки	39
4.9	Анализ устойчивости стратегий	40
4.10	Ограничения проведенного исследования	40
4.11	Выводы по главе	41
	ЗАКЛЮЧЕНИЕ	42
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	45
	ПРИЛОЖЕНИЕ А	48

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе применяют следующие термины с соответствующими определениями:

Биржевая котировка — опубликованное во времени значение цены финансового инструмента, сформированное в результате биржевых сделок и используемое для анализа рыночной динамики

Финансовый временной ряд — упорядоченная по времени последовательность наблюдений за рыночными переменными, такими как цена открытия, максимум, минимум, цена закрытия и объем торгов

OHLCV-данные — формат представления рыночной информации, включающий цену открытия (*Open*), максимум (*High*), минимум (*Low*), цену закрытия (*Close*) и объем торгов (*Volume*)

Торговая стратегия — формализованный набор правил, определяющий условия открытия, удержания и закрытия позиции на финансовом рынке

Baseline-стратегия — простая эталонная стратегия, используемая как нижняя граница качества при сравнении более сложных алгоритмов

Признак — числовая характеристика состояния рынка, используемая моделью для принятия решения или прогнозирования

Целевая переменная — величина, которую модель должна предсказать, например направление изменения цены или доходность следующего периода

Обучение с подкреплением — парадигма машинного обучения, при которой агент последовательно взаимодействует со средой, выбирает действия и получает награду за последствия этих действий

Агент — обучаемая система, принимающая решения в среде на основе наблюдаемого состояния

Состояние среды — совокупность признаков, описывающих доступную агенту информацию о рынке и текущем состоянии портфеля

Функция вознаграждения — правило количественной оценки результата действия агента, используемое для настройки его поведения

Equity curve — кривая стоимости портфеля во времени, отражающая изменение капитала при выполнении стратегии

Просадка — относительное снижение стоимости портфеля от локального максимума до последующего минимума

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе применяют следующие сокращения и обозначения:

ИИ — искусственный интеллект

МО — машинное обучение

RL — reinforcement learning, обучение с подкреплением

DL — deep learning, глубокое обучение

OHLCV — open, high, low, close, volume

SMA — simple moving average, простое скользящее среднее

RSI — relative strength index

MACD — moving average convergence divergence

ARIMA — autoregressive integrated moving average

SARIMA — seasonal autoregressive integrated moving average

LSTM — long short-term memory

GRU — gated recurrent unit

DQN — deep Q-network

PPO — proximal policy optimization

A2C — advantage actor-critic

ROC-AUC — area under the receiver operating characteristic curve

MDP — Markov decision process

API — application programming interface

ВВЕДЕНИЕ

Современные финансовые рынки характеризуются высокой скоростью обновления информации, существенной изменчивостью цен и постоянной конкуренцией между участниками, использующими различные аналитические и вычислительные методы. В этой среде задача прогнозирования биржевых котировок остается одной из ключевых как для прикладной финансовой аналитики, так и для области искусственного интеллекта. Однако накопленный опыт показывает, что сама по себе задача прогноза цены не исчерпывает практическую постановку проблемы. Даже при наличии модели, демонстрирующей приемлемое качество по статистическим критериям, ее полезность для принятия торговых решений может оказаться ограниченной. Модель может корректно предсказывать направление части движений, но при этом формировать чрезмерно частые сигналы, создавать высокую просадку или не учитывать транзакционные издержки [3, 6, 7].

Актуальность темы определяется сразу несколькими обстоятельствами. Во-первых, финансовые временные ряды являются типичным примером сложных нестационарных данных, для которых недостаточно только классических статистических методов. Во-вторых, широкое распространение получили модели машинного обучения и глубокого обучения, ориентированные на выделение сложных зависимостей в данных. В-третьих, в последние годы заметно вырос интерес к обучению с подкреплением как к подходу, который позволяет не просто прогнозировать цену, а обучать агента непосредственному выбору действий на рынке. Для задач алгоритмической торговли это особенно важно, поскольку конечная цель практической системы состоит не в минимизации ошибки прогноза как таковой, а в повышении эффективности последовательных торговых решений с учетом риска, просадки, комиссий и устойчивости поведения на различных рыночных режимах [3, 7, 11, 12, 21].

Дополнительную значимость теме придает замечание, сделанное научным руководителем: при обосновании актуальности необходимо не только описать применимость RL-подходов, но и провести сравнение существующих аналогов, а также четко показать проблему, которую решает работа. Действительно, без такого сравнения исследование рискует превратиться в изолированное описание одного алгоритма. Поэтому в настоящей работе обучение с подкреплением рассматривается не как самоцель, а как один из клас-

сов методов, сопоставляемый с традиционными baseline-стратегиями, статистическими моделями и классическими алгоритмами машинного обучения в рамках единого экспериментального стенда.

Проблема исследования состоит в следующем. Большинство подходов к анализу биржевых данных оцениваются либо по ошибке прогноза, либо по отдельным финансовым показателям, но не всегда в рамках единой методологии. На практике это приводит к нескольким методологическим трудностям. Во-первых, сравнение моделей оказывается некорректным, если они обучаются и тестируются на разных интервалах или по разным правилам формирования сигналов. Во-вторых, высокая классификационная точность не эквивалентна высокой доходности стратегии, поскольку итоговый результат зависит от частоты сделок, величины прибыли на одну сделку, транзакционных издержек и характера рыночного режима. В-третьих, многие работы рассматривают RL-агентов обособленно, без сопоставления с простыми и интерпретируемыми альтернативами, такими как Buy & Hold или стратегия на пересечении скользящих средних [6, 7, 27, 28].

Таким образом, возникает необходимость в разработке программного решения, которое позволяет на едином наборе биржевых данных формировать признаки, обучать модели, запускать алгоритмы принятия торговых решений, выполнять backtesting и рассчитывать показатели качества по единым правилам. Именно создание и экспериментальная проверка такого решения составляют центральное содержание настоящей выпускной квалификационной работы.

Целью работы является разработка, программная реализация и экспериментальная проверка программного решения для прогнозирования и оптимизации торговых решений на основе биржевых котировок с использованием baseline-стратегий, моделей машинного обучения и методов обучения с подкреплением.

Для достижения поставленной цели в работе решаются следующие задачи:

- 1) проанализировать особенности биржевых котировок как финансовых временных рядов и определить требования к разрабатываемому программному решению;
- 2) выполнить обзор существующих подходов и аналогов, необходимых

для выбора состава реализуемых модулей и методов сравнения;

- 3) спроектировать архитектуру программного решения, включающего загрузку данных, формирование признаков, модуль принятия решений и подсистему оценки результатов;
- 4) реализовать модуль подготовки OHLCV-данных, модуль генерации признаков, baseline-стратегии, классические ML-модели и RL-среду для обучения агентов DQN и PPO;
- 5) формализовать задачу обучения с подкреплением, включая описание состояния среды, пространства действий, функции вознаграждения и ограничений торговой логики;
- 6) реализовать процедуры обучения, тестирования и backtesting, обеспечивающие воспроизводимый запуск вычислительного эксперимента;
- 7) провести экспериментальную проверку разработанного программного решения и сравнить реализованные подходы по классификационным и финансовым метрикам;
- 8) выполнить анализ полученных результатов, определить ограничения решения и наметить направления его дальнейшего развития.

Объектом исследования являются методы анализа и принятия решений на основе биржевых котировок.

Предметом исследования выступают алгоритмы прогнозирования и оптимизации торговых стратегий на финансовых временных рядах, в том числе baseline-подходы, модели машинного обучения и методы обучения с подкреплением.

В работе применяются методы статистического анализа временных рядов, методы машинного обучения, методы обучения с подкреплением, методы финансовой оценки торговых стратегий, а также методы программной инженерии для построения воспроизводимого вычислительного контура. Практическая реализация выполнена на языке Python с использованием библиотек *pandas*, *NumPy*, *scikit-learn*, *yfinance*, *Gymnasium*, *stable-baselines3* и *matplotlib* [15, 16, 17, 19, 20, 24].

Практическая значимость работы заключается в создании работоспособного программного решения, пригодного для воспроизводимого запуска экспериментов по алгоритмической торговле на исторических котировках. Разработанный pipeline позволяет автоматически загружать данные, формировать признаки, обучать реализованные модели, запускать стратегии на тестовом интервале, рассчитывать финансовые показатели и сохранять результаты в виде отчетных артефактов. Такой подход представляет интерес как для учебных целей, так и для дальнейшего наращивания функциональности за счет новых признаков, алгоритмов и сценариев оценки.

Научная новизна работы в рамках учебного исследования состоит в том, что RL-подход рассматривается не изолированно, а как часть разработанного программного контура, в котором прогнозирующие модели и торговые стратегии сравниваются по единым правилам входных данных, тестирования и расчета метрик. Это позволяет сместить акцент с абстрактного качества прогноза на более прикладной вопрос: может ли реализованный RL-агент улучшить не только точность сигнала, но и риск-скорректированную результативность торговой системы.

Структурно работа включает введение, четыре главы, заключение, список использованных источников и приложение. В первой главе рассматриваются предметная область, существующие подходы и аналоги, а также формулируется проблема исследования. Во второй главе описываются проектирование метода, подготовка данных, признаки, постановка RL-задачи и система метрик. Третья глава посвящена реализации программного решения и экспериментального стенда. В четвертой главе представлены результаты экспериментального моделирования, их сопоставление и интерпретация. В заключении сформулированы основные результаты, ограничения и направления дальнейшего развития работы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ АНАЛОГОВ

1.1 Особенности биржевых котировок как временных рядов

Биржевые котировки относятся к классу финансовых временных рядов, для которых характерно сочетание трендовых участков, шумовой компоненты, изменения волатильности и зависимости от внешних новостных и макроэкономических факторов. В отличие от многих инженерных временных рядов, в рыночных данных редко выполняется предположение о строгой стационарности. Распределение доходностей может меняться во времени, амплитуда колебаний возрастает в периоды стрессов, а закономерности, найденные на одном интервале, часто деградируют на другом. Это явление в прикладных работах связывают с изменением рыночного режима и эффектом *concept drift* [3, 25, 26].

С практической точки зрения биржевой ряд представляет собой не просто последовательность значений цены, а комплексную динамическую систему, где существенную роль играют доходности, скорость изменения цены, локальный тренд, объемы торгов и взаимодействие нескольких временных масштабов. Для целей построения торговых стратегий важны не только ожидаемый уровень цены, но и вероятность благоприятного движения в ближайшем будущем, вероятность ложного сигнала, длительность удержания позиции и ожидаемая величина просадки. Вследствие этого выбор модели должен учитывать как прогностическую силу признаков, так и последующее влияние предсказаний на торговый контур [3, 6].

Дополнительная сложность заключается в том, что на финансовых рынках высокая ошибка в несколько отдельных моментов способна перечеркнуть полезность большого числа корректных прогнозов. Например, стратегия может правильно предсказывать большую долю небольших движений, но потерять значительную часть капитала на редких сильных разворотах. Поэтому интерпретация качества модели через одну лишь среднюю ошибку или точность классификации является неполной. Для корректного исследования необходимо переходить от анализа ряда как статистического объекта к анализу результата как динамики стоимости портфеля [7, 27, 28].

1.2 Классические методы прогнозирования финансовых временных рядов

Классические методы прогнозирования финансовых временных рядов включают наивные стратегии, модели сглаживания, авторегрессионные модели и сезонные модификации, такие как ARIMA и SARIMA. Их достоинство состоит в хорошо разработанном математическом аппарате, интерпретируемости параметров и понятной процедуре оценки. Для многих экономических и производственных рядов такие модели остаются надежным инструментом. Однако в отношении биржевых котировок их применимость ограничивается тем, что динамика цен часто содержит выраженную нелинейность, структурные сдвиги и слабую устойчивость локальных закономерностей [2, 4].

Наивная модель *random walk*, предполагающая, что лучшее предсказание следующего значения равно текущему, долгое время использовалась как базовая точка отсчета для оценки более сложных алгоритмов. В рамках торговли ей соответствует *Buy & Hold*, если актив находится в долгосрочном восходящем тренде, или пассивное отсутствие позиции, если цель исследования сводится к прогнозированию самого уровня цены. Скользящие средние, экспоненциальное сглаживание и ARIMA-подходы позволяют выделять локальные зависимости, но плохо переносят резкие изменения режима, характерные для финансовых рынков [4, 25, 26].

В контексте настоящей работы классические статистические модели важны по двум причинам. Во-первых, они задают нижнюю и среднюю планку для сравнения более современных методов. Во-вторых, они демонстрируют ограничение постановки, в которой цель исследования сводится исключительно к прогнозу следующего значения ряда. Даже хорошо подогнанная ARIMA-модель не определяет напрямую, когда открывать позицию, как долго ее удерживать и как учитывать цену ошибки в терминах реального капитала [2, 4, 6].

1.3 Методы машинного обучения для прогнозирования котировок

В отличие от классических статистических моделей, методы машинного обучения ориентированы на работу с признаковым представлением задачи. Входом для модели становится не сам ряд в исходном виде, а набор признаков, включающий лаги, доходности, скользящие средние, индикато-

ры импульса, оценки волатильности и другие характеристики. Такой переход позволяет формулировать задачу как бинарную классификацию направления следующего движения или как регрессию доходности [7, 15].

Для задач табличного прогнозирования биржевых котировок особенно распространены *Logistic Regression*, *Random Forest*, *Gradient Boosting*, а также продвинутые реализации бустинга, такие как CatBoost и XGBoost. Логистическая регрессия обеспечивает высокую интерпретируемость и позволяет быстро получить базовый ориентир качества. Ансамблевые методы, в свою очередь, способны захватывать нелинейные зависимости между признаками и часто демонстрируют более устойчивый результат на шумных данных [13, 14].

Однако переход к машинному обучению не устраняет фундаментальную проблему исследования: прогноз направления сам по себе не гарантирует построение эффективной стратегии. Одна и та же модель может давать приемлемый ROC-AUC, но формировать слишком частые сигналы, приводящие к избыточным комиссиям, либо слабо работать в периоды рыночного боковика. Поэтому в работе ML-модели рассматриваются не как конечный инструмент принятия решений, а как промежуточный источник торгового сигнала, который затем преобразуется в стратегию и оценивается уже по финансовым метрикам [6, 7].

1.4 Нейросетевые модели для анализа временных рядов

Нейросетевые модели получили широкое распространение в задачах анализа временных рядов благодаря способности автоматически извлекать сложные нелинейные закономерности из последовательностей данных. Для финансовых задач наиболее часто используются LSTM и GRU, а также их модификации, комбинируемые с attention-механизмами, сверточными блоками или трансформерными архитектурами. Преимущество таких моделей состоит в том, что они потенциально могут учитывать более длинный контекст, чем классические табличные модели [11, 12].

Тем не менее в прикладных задачах алгоритмической торговли нейросетевые подходы нередко сталкиваются с рядом проблем. Во-первых, они требуют более аккуратной настройки гиперпараметров и, как правило, большего объема данных. Во-вторых, их устойчивость на нестационарных финан-

совых рядах не гарантирована: модель может хорошо описывать один временной режим и деградировать после его смены. В-третьих, интерпретируемость результатов оказывается заметно ниже, чем у линейных и ансамблевых моделей [7, 11, 12].

В настоящей работе LSTM и GRU рассматриваются как важные аналоги в теоретическом обзоре, поскольку они занимают промежуточное положение между классическим ML и RL-подходами. Они по-прежнему ориентированы на прогноз будущего значения или класса, но делают это в более гибкой нелинейной форме. Однако основной экспериментальный контур работы сфокусирован на baseline-стратегиях, трех ML-моделях и RL-агентах, что обусловлено необходимостью сохранить воспроизводимость и умеренную вычислительную сложность на локальной вычислительной платформе [11, 12].

1.5 Обучение с подкреплением в задачах алгоритмической торговли

Обучение с подкреплением принципиально отличается от классического прогностического подхода тем, что объектом оптимизации является не ошибка прогноза, а последовательность действий агента в среде. В задачах алгоритмической торговли это особенно важно: вместо предсказания отдельного значения цены агент получает состояние рынка, выбирает действие и получает награду, связанную с изменением стоимости портфеля, комиссией и риском. Таким образом, модель обучается непосредственно в терминах поведения, а не только в терминах прогноза [1, 6, 8, 9].

Наиболее известные RL-алгоритмы, применяемые в торговых задачах, можно разделить на value-based и policy-based семейства. DQN относится к первой группе и аппроксимирует функцию ценности действий, что удобно для дискретных действий *buy*, *sell*, *hold*. PPO относится ко второй группе и обучает параметризованную политику с ограничением шага обновления, обеспечивая более устойчивую оптимизацию. A2C сочетает идеи актор-критик и служит компактной альтернативой при меньших вычислительных затратах [8, 9, 10].

Несмотря на привлекательность RL, его использование в финансовых задачах не гарантирует автоматического превосходства над более простыми подходами. Качество RL-агента крайне зависит от дизайна среды, наборо-

ра признаков, функции вознаграждения и выбранного рыночного режима. Именно поэтому RL в настоящем исследовании оценивается не как заведомо лучший метод, а как один из классов алгоритмов, потенциально полезный для оптимизации последовательных решений, но требующий строгого экспериментального сопоставления с альтернативами [1, 8, 9].

1.6 Обзор аналогов и существующих решений: FinRL, TensorTrade, gym-anytrading

Среди готовых исследовательских и прикладных решений для RL-трейдинга можно выделить платформы FinRL, TensorTrade и библиотеку gym-anytrading. FinRL предоставляет набор стандартных сред, примеров и сценариев обучения RL-агентов на финансовых данных. Его достоинство состоит в ориентации на воспроизводимость и наличии типовых пайплайнов для получения котировок, подготовки данных и запуска алгоритмов. Однако при этом исследователь во многом зависит от архитектурных решений, заложенных в сам фреймворк [21].

TensorTrade ориентирован на более модульный подход. В нем торговая среда строится из взаимодействующих компонентов: источника данных, механизма исполнения, портфеля, действия и функции награды. Такой подход обеспечивает гибкость, но одновременно повышает порог входа и усложняет тонкую настройку исследования. Для учебной и дипломной работы это может привести к значительным трудозатратам на инфраструктурный уровень вместо концентрации на собственно методологическом сравнении [22].

Библиотека gym-anytrading занимает противоположную позицию. Она предлагает минималистичные среды, совместимые с Gym-интерфейсом, что удобно для быстрого прототипирования. Ее слабой стороной является ограниченная выразительность модели рынка и меньшее число готовых сценариев для серьезного экспериментального расширения. По этой причине в настоящей работе сделан выбор в пользу собственной среды на базе Gymnasium: она сохраняет совместимость со stable-baselines3, но при этом остается достаточно прозрачной и контролируемой для дипломного исследования [20, 23].

1.7 Сравнительная таблица аналогов

Для систематизации рассмотренных подходов в таблице 1 приведено сравнительное описание стратегий, моделей и фреймворков, имеющих наибольшее значение для поставленной задачи. Таблица показывает, что ни один из подходов не является универсальным. Простые baseline-стратегии обеспечивают максимальную интерпретируемость, статистические модели подходят для формального анализа временных рядов, ML-модели хорошо работают на табличных признаках, а RL-алгоритмы наиболее естественно описывают последовательное принятие торговых решений.

Таблица 1 — Сравнение существующих аналогов и подходов

Подход / средство	Класс	Сильные стороны	Ограничения	Роль в исследовании
Buy & Hold	baseline-стратегия	Простота, прозрачность, отражение рыночного бета-эффекта	Не адаптируется к рыночным режимам, не управляет риском активно	Базовый эталон абсолютной доходности
Moving Average Crossover	техническая стратегия	Интерпретируемость, учет тренда, низкая вычислительная сложность	Запаздывание сигнала, чувствительность к параметрам окна, ложные входы на боковом рынке	Сопоставление RL с простым правилом на индикаторах
ARIMA / SARIMA	статистическое прогнозирование	Формальный аппарат, интерпретируемые параметры, удобство для линейной динамики	Плохо описывает нелинейность и смену режимов, требует стационаризации	Аналог прогнозирования ценового ряда
Logistic Regression	классическое МО	Простота, устойчивость, возможность анализа вклада признаков	Ограниченная нелинейность, зависимость от качества признаков	Базовая ML-модель для бинарного прогноза направления
Random Forest	ансамблевое МО	Работа с нелинейными зависимостями, устойчивость к шуму, малая потребность в масштабировании	Снижение интерпретируемости, риск переобучения при слабой валидации	Сравнение с линейной моделью и RL

Подход / средство	Класс	Сильные стороны	Ограничения	Роль в исследовании
Gradient Boosting	ансамблевое МО	Высокая гибкость, качественная аппроксимация сложных закономерностей	Требовательность к настройке, чувствительность к качеству данных	Сильный ML-базис в экспериментальном стенде
CatBoost / XGBoost	градиентный бустинг	Высокое качество на табличных данных, эффективная работа с признаками	Выше вычислительные затраты, необходимость тщательного тюнинга	Аналог продвинутого табличного бустинга
LSTM / GRU	нейросетевые модели	Учет последовательной структуры, моделирование временной зависимости	Чувствительность к объему данных, сложность настройки, риск нестабильного обучения	Аналоги глубокого обучения для временных рядов
DQN	RL-алгоритм value-based	Обучение дискретным действиям, прямое связывание состояния и торгового решения	Нестабильность при плохой reward function, чувствительность к коррелированным данным	RL-аналог с дискретным пространством действий
PPO	RL-алгоритм policy-based	Устойчивое обновление политики, хорошая практическая применимость, удобство для continuous optimization	Более высокая стоимость обучения, зависимость от дизайна среды	Основной RL-алгоритм для сравнения риск-скорректированной эффективности
A2C	actor-critic	Более компактная альтернатива PPO, обучает политику и value function совместно	Меньшая устойчивость по сравнению с PPO на шумных рынках	Рассматривается как альтернативный RL-подход в обзоре
FinRL	фреймворк	Готовые среды, примеры RL-трейдинга, акцент на reproducibility	Сильная зависимость от встроенных сценариев, сложность адаптации под конкретный pipeline	Аналог исследовательской платформы

Подход / средство	Класс	Сильные стороны	Ограничения	Роль в исследовании
TensorTrade	фреймворк	Модульная архитектура, абстракции обмена, портфеля и вознаграждения	Повышенный порог входа, необходимость глубокой настройки	Аналог конструкторного инструментария
gym-anytrading	библиотека сред	Простота, совместимость с Gym, быстрое прототипирование	Ограниченная гибкость и сравнительно простая рыночная логика	Аналог минималистичной RL-среды

Дополнительно в таблице 2 показано, что принципиальное различие между классами методов состоит в типе целевого результата и в критерии качества. Для baseline- и RL-подходов естественной метрикой является динамика капитала, тогда как для статистических, ML- и нейросетевых моделей часто сначала оценивается ошибка прогноза, а уже затем финансовая результативность стратегии, построенной на основе этого прогноза.

Таблица 2 — Сопоставление классов методов, входов и критериев оценки

Группа методов	Входные данные	Целевой результат	Основной критерий оценки
Baseline-стратегии	Цена закрытия, скользящие средние	Последовательность торговых сигналов по фиксированному правилу	Финансовые метрики стратегии
Статистические модели	Ряд цен или доходностей	Прогноз следующего значения или доходности	Ошибка прогноза и прибыльность стратегии на основе сигнала
Классические ML-модели	Табличные признаки на основе OHLCV и индикаторов	Класс направления движения или доходность следующего периода	ML-метрики и финансовые метрики после конвертации прогноза в стратегию
Нейросетевые модели	Последовательности наблюдений, лаги, индикаторы	Прогноз направления, уровня цены или доходности	Ошибка прогноза, устойчивость и финансовый результат
RL-агенты	Окно признаков, позиция, доля cash	Действие buy / sell / hold	Накопленная награда и финансовые метрики стратегии

1.8 Постановка проблемы и выводы по главе

Проведенный анализ предметной области и аналогов позволяет сформулировать центральную проблему исследования более точно. Существующие работы и инструменты часто сосредоточены либо на точности прогноза, либо на отдельных аспектах торговой результативности. Однако для практической задачи алгоритмической торговли этого недостаточно. Высокая точность классификации не тождественна прибыльности стратегии, а высокая доходность на одном интервале не гарантирует устойчивости на другом. Следовательно, требуется единый экспериментальный стенд, который позволяет

сравнивать baseline-стратегии, модели машинного обучения и RL-агентов не изолированно, а в одном методическом контуре.

В рамках такой постановки работа решает именно проблему корректного сравнительного анализа. Она не стремится доказать, что RL всегда превосходит классические подходы. Напротив, исследовательская гипотеза формулируется осторожно: RL может быть полезен тогда, когда задача требует оптимизировать последовательность действий и балансировать между доходностью и риском, однако его реальная ценность должна подтверждаться сравнением с простыми и интерпретируемыми альтернативами. Такой вывод задает логический переход к следующей главе, в которой описываются проектирование метода и построение экспериментального стенда для проведения указанного сравнения.

2 ПРОЕКТИРОВАНИЕ МЕТОДА И ЭКСПЕРИМЕНТАЛЬНОГО СТЕНДА

2.1 Общая архитектура исследования

В данной работе реализуется не абстрактная схема сравнения методов, а конкретное программное решение, предназначенное для прогнозирования и оптимизации торговых решений на исторических биржевых котировках. Архитектура решения строится как последовательный pipeline обработки данных и принятия решений, в котором каждый модуль имеет четко определенные входы и выходы. Такой подход необходим для того, чтобы все сравниваемые методы находились в одинаковых условиях: использовали одну и ту же исходную информацию, одинаковую схему разбиения данных и единый набор метрик.

Архитектурно программное решение состоит из следующих взаимосвязанных компонентов:

- 1) модуль загрузки и предобработки биржевых данных, формирующий исходный набор OHLCV-наблюдений;
- 2) модуль формирования признаков, преобразующий ряд цен в набор технических и статистических характеристик;
- 3) модуль baseline-стратегий и ML-моделей, генерирующий торговые сигналы на основе фиксированных правил или прогноза направления движения;
- 4) среда обучения с подкреплением, описывающая состояние рынка, действия агента и правила обновления портфеля;
- 5) RL-агент, обучаемый принимать решения *buy*, *sell*, *hold* на основе наблюдаемого состояния;
- 6) модуль обучения и оркестрации эксперимента, запускающий подготовку данных, обучение, тестирование и сохранение артефактов;
- 7) модуль тестирования и backtesting, преобразующий сигналы и действия в траекторию стоимости портфеля;

- 8) модуль расчета метрик и визуализации результатов, формирующий таблицы, графики и итоговые сравнительные отчеты.

На рисунке 1 показана логика взаимодействия модулей. Важной особенностью стенда является то, что финансовые метрики выносятся в общий слой оценки. Это позволяет сравнивать модели, исходно обучавшиеся по разным критериям. Для ML-моделей такой критерий связан с качеством классификации, а для RL-моделей — с накопленной наградой. Тем не менее итоговое сравнение выполняется в одном пространстве показателей доходности и риска.

Поток данных в разработанном решении имеет следующий вид: исторические котировки поступают в модуль предобработки, затем на их основе формируется пространство признаков, после чего эти признаки либо подаются в baseline- и ML-модели, либо используются для построения состояния RL-среды. Далее агент или модель формирует торговое действие, на основе которого обновляется портфель, рассчитывается награда и строится стратегия на тестовом интервале. После завершения тестирования рассчитываются финансовые метрики и создаются графические артефакты, позволяющие интерпретировать результат.

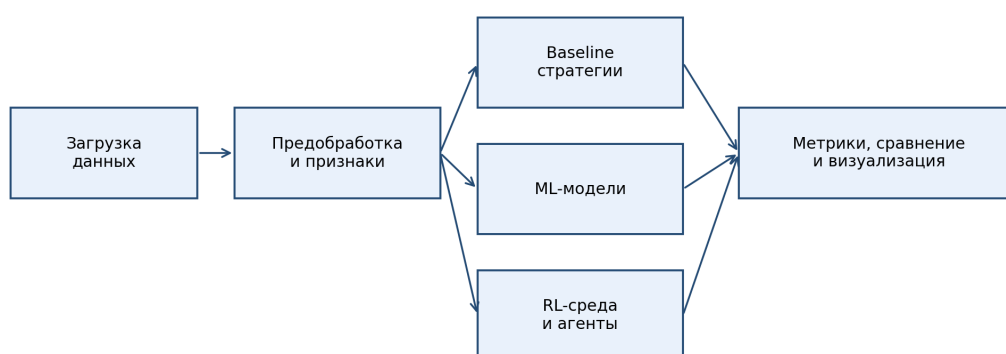


Рис. 1 — Архитектура программного решения для прогнозирования и оптимизации торговых решений на основе RL

2.2 Источники данных и структура OHLCV-данных

В качестве источника рыночных данных в работе выбран сервис Yahoo Finance, доступ к которому осуществляется через библиотеку *yfinance*. Такой выбор обусловлен несколькими причинами. Во-первых, источник

обеспечивает достаточно удобный программный доступ к дневным котировкам широкого круга инструментов. Во-вторых, формат данных легко интегрируется в Python-пайплайн без промежуточных ручных преобразований. В-третьих, полученные данные пригодны для воспроизводимых учебно-исследовательских экспериментов [24].

Базовой единицей наблюдения является строка формата OHLCV, содержащая дату, цену открытия, максимум, минимум, цену закрытия, скорректированную цену закрытия и объем. Для торговых задач цена закрытия используется как основа расчета доходностей и большинства технических индикаторов. Скорректированная цена закрытия позволяет учитывать дивиденды и сплиты в тех сценариях, где это критично для корректного сравнения исторических данных [5, 24].

Использование OHLCV-данных удобно и с точки зрения расширяемости исследования. Даже если в базовом эксперименте часть признаков вычисляется только по цене закрытия, наличие полного OHLCV-набора позволяет в дальнейшем добавлять диапазонные характеристики, свечные паттерны, индикаторы объема и более сложные рыночные признаки.

2.3 Предобработка данных

Предобработка данных включает несколько обязательных этапов. Сначала выполняется проверка полноты схемы и корректности дат. Затем данные сортируются в хронологическом порядке и очищаются от строк с пропусками в критичных колонках. После этого производится приведение числовых полей к единому формату и отбрасывание промежуточных значений, способных нарушить воспроизводимость результатов.

Следующим важным шагом является разбиение данных на обучающую, валидационную и тестовую части без перемешивания. Для финансовых временных рядов это принципиально важно, поскольку случайное перемешивание приводит к утечке информации из будущего в прошлое. В настоящей работе используется временное разбиение, при котором сначала выделяется обучающий интервал, затем валидационный, а затем тестовый период. Такое разбиение ближе к реальному сценарию, где модель всегда обучается на исторических данных и применяется к более позднему участку ряда [2, 3, 7].

Отдельно необходимо отметить роль транзакционных издержек. Даже если в исходном ряду цены отражают рыночное движение корректно, стратегия, активно меняющая позицию, может терять значительную часть результата на комиссии. Поэтому уже на этапе проектирования стенда в торговую логику baseline-стратегий и RL-среды закладывается параметр *transaction_cost*, позволяющий избежать чрезмерно оптимистичных оценок [6, 7].

2.4 Формирование признаков: доходности, скользящие средние, RSI, MACD, Bollinger Bands, волатильность

Для преобразования исходных OHLCV-данных в признаки, пригодные для обучения моделей, используется набор технических и статистических характеристик, отражающих как локальную доходность, так и структуру тренда и риска. Базовыми признаками являются простая доходность и логарифмическая доходность [3, 5, 7]:

$$r_t = \frac{C_t}{C_{t-1}} - 1, \quad (1)$$

$$\ell_t = \ln \left(\frac{C_t}{C_{t-1}} \right), \quad (2)$$

где C_t — цена закрытия в момент времени t .

Для описания трендовой компоненты используются скользящие средние с окнами 5, 10 и 20 периодов:

$$\text{SMA}_n(t) = \frac{1}{n} \sum_{i=0}^{n-1} C_{t-i}. \quad (3)$$

Такой набор позволяет одновременно учитывать краткосрочную и среднесрочную динамику. Для оценки риска добавляются скользящие стандартные отклонения доходности на окнах 10 и 20 периодов, характеризующие локальную волатильность.

Индикатор RSI используется для оценки локальной перекупленности и перепроданности. MACD и его сигнальная линия позволяют отслеживать расхождение между быстрыми и медленными экспоненциальными средними, а полосы Боллинджера помогают оценить отклонение текущей цены от

локального среднего уровня с учетом волатильности. Выбор именно этих признаков обусловлен их широкой представленностью в практических торговых исследованиях и достаточной интерпретируемостью [5, 7].

Все признаки рассчитываются только на исторической информации, доступной на текущем шаге. После генерации признаки проходят очистку от значений *NaN*, возникающих из-за оконных вычислений. Благодаря этому обучающие алгоритмы получают согласованную матрицу признаков без утечки информации из будущего.

2.5 Формирование целевых переменных для ML-моделей

Для классических моделей машинного обучения в работе используются две целевые переменные. Первая — *target_direction*, представляющая собой бинарный индикатор направления изменения цены на следующем шаге [7]:

$$y_t = \begin{cases} 1, & \text{если } C_{t+1} > C_t, \\ 0, & \text{иначе.} \end{cases} \quad (4)$$

Эта постановка удобна для задач классификации и позволяет оценивать качество моделей по стандартным ML-метрикам.

Вторая целевая переменная — *target_return*, то есть доходность следующего периода:

$$\rho_t = \frac{C_{t+1}}{C_t} - 1. \quad (5)$$

Хотя в основном экспериментальном контуре работа сосредоточена на классификации направления движения, использование доходности в качестве дополнительной переменной важно для интерпретации результата. Во-первых, она позволяет перевести прогноз модели в торговый эффект. Во-вторых, ее наличие упрощает последующую конвертацию предсказаний в стратегию и расчет совокупной доходности.

2.6 Baseline-стратегии: Buy & Hold и Moving Average Crossover

В качестве простейших аналогов в стенд включены две baseline-стратегии. Первая стратегия — *Buy & Hold* — предполагает покупку актива в начале тестового интервала и удержание позиции до его завершения. Эта стратегия особенно важна в задачах анализа фондового рынка, поскольку

на долгосрочных восходящих участках она может обеспечивать высокий абсолютный результат при минимальной сложности реализации. В то же время *Buy & Hold* почти не управляет риском и не адаптируется к смене рынка [5, 6].

Вторая стратегия — *Moving Average Crossover* — использует пересечение короткой и длинной скользящих средних. Если короткая средняя находится выше длинной, стратегия считается находящейся в длинной позиции; иначе позиция закрывается. Такой подход отражает распространенную практику использования трендовых индикаторов и служит промежуточным этапом между пассивной стратегией и моделями, обучаемыми на данных [5, 6].

Принципиально важно, что обе стратегии оцениваются по тем же финансовым показателям, что и ML- и RL-подходы. Это позволяет интерпретировать выигрыш сложной модели не в абстрактных терминах, а через конкретное улучшение доходности, просадки или коэффициента Шарпа относительно понятных и прозрачных альтернатив.

2.7 Постановка задачи обучения с подкреплением

Для RL-части исследования задача формулируется как задача последовательного принятия решений в среде, описываемой процессом принятия решений Маркова. На вход среды поступает *DataFrame* с историческими OHLCV-данными и рассчитанными признаками, а также список колонок, формирующих наблюдаемое состояние. На каждом шаге агент наблюдает состояние среды, выбирает одно из допустимых действий и получает награду, зависящую от изменения стоимости портфеля и комиссий за сделки. Таким образом, целью оптимизации становится не качество одношагового прогноза, а максимизация накопленного полезного результата на всем тестовом интервале [1].

В настоящей работе рассматривается long-only-постановка с дискретными действиями *hold*, *buy* и *sell*. Такое ограничение упрощает интерпретацию и делает пространство решений совместимым с DQN-алгоритмом. Одновременно эта же среда может использоваться с PPO, что обеспечивает сопоставимость value-based и policy-based подходов [8, 9]. В реализованной среде агент не открывает короткие позиции, а операция *buy* использует весь

доступный денежный остаток, тогда как операция *sell* полностью закрывает текущую длинную позицию.

Выбор именно такой постановки продиктован прикладной логикой разработанного прототипа. Во многих прикладных задачах достаточно ответить на вопрос, стоит ли в данный момент сохранять позицию, открыть ее или выйти в кэш. Даже такой ограниченный набор действий уже позволяет оценить, в какой степени RL-агент способен использовать признаки рынка для адаптивной торговли и дает возможность сравнить его с более простыми реализованными альтернативами.

2.8 Описание состояния, действий и функции вознаграждения RL-агента

Состояние RL-агента формируется как окно последних *window_size* наблюдений по выбранным признакам, дополненное информацией о текущей позиции и доле капитала, находящейся в наличных средствах. В реализованной версии среды *window_size* по умолчанию равно 30, а в набор признаков входят *simple_return*, *log_return*, *rolling_mean_5*, *rolling_mean_10*, *rolling_mean_20*, *rolling_std_10*, *rolling_std_20*, *rsi_14*, *macd*, *macd_signal*, *bollinger_upper* и *bollinger_lower*. Такой вектор состояния совмещает сведения о локальной рыночной динамике и внутреннем состоянии портфеля. Агент не получает доступа к будущим значениям и действует только на основе исторической информации [1, 8, 9].

Как показано на рисунке 2, действие *buy* переводит свободный капитал в позицию по активу, действие *sell* закрывает позицию и возвращает капитал в денежную форму, а действие *hold* сохраняет текущую структуру портфеля. Формально пространство действий можно описать следующим образом: 0 — удержание текущего состояния портфеля, 1 — покупка актива на весь доступный объем денежных средств, 2 — продажа всей текущей позиции. Вознаграждение определяется через изменение *portfolio_value* между соседними шагами с учетом уплаченной комиссии. Такой выбор *reward function* делает обучение более прикладным, поскольку агент оптимизирует непосредственно то, что имеет финансовый смысл.

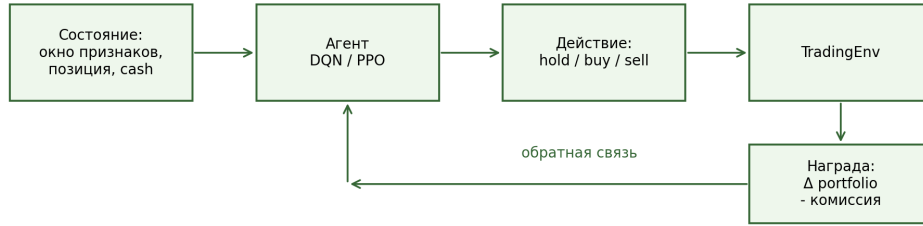


Рис. 2 — Схема взаимодействия RL-агента со средой

При этом следует учитывать, что даже разумно выбранная функция вознаграждения не гарантирует универсально качественного поведения. Если агент получает награду только за локальный прирост капитала, он может оказаться склонен к чрезмерной активности. Если, напротив, reward чрезмерно штрафует риск, агент может выбирать слишком осторожную политику. Поэтому в дипломной работе RL рассматривается как чувствительный к постановке задачи метод, а не как универсальная замена прогнозным моделям. Ограничениями разработанного решения на данном этапе являются long-only-логика, торговля одним активом в каждый момент времени, фиксированная модель комиссии и отсутствие моделирования проскальзывания.

2.9 Метрики оценки качества моделей и стратегий

Поскольку в стенде сравниваются модели разной природы, для оценки используются две группы метрик: классификационные и финансовые. Классификационные метрики — Accuracy, Precision, Recall, F1-score и ROC-AUC — применяются для ML-моделей, прогнозирующих направление движения. Они позволяют понять, насколько корректно модель различает положительный и отрицательный классы. Однако сами по себе эти показатели не отражают экономический результат использования модели [15].

Финансовые метрики, напротив, оценивают уже поведение торговой стратегии. Совокупная доходность определяется как [27, 28]:

$$R_{\text{total}} = \prod_{t=1}^T (1 + r_t^{(s)}) - 1, \quad (6)$$

где $r_t^{(s)}$ — доходность стратегии в период t . Годовая волатильность, коэффициент Шарпа и максимальная просадка позволяют оценить риск, устойчивость и глубину неблагоприятных периодов:

$$\text{Sharpe} = \frac{\overline{r^{(s)}} - r_f}{\sigma(r^{(s)})} \sqrt{K}, \quad (7)$$

где r_f — безрисковая ставка, $\sigma(r^{(s)})$ — стандартное отклонение доходности стратегии, K — число периодов в году [27].

Таблица 3 — Метрики, применяемые в исследовании

Метрика	Тип	Интерпретация
Accuracy	ML	Доля правильно классифицированных направлений движения цены
Precision	ML	Доля верных положительных сигналов среди всех положительных прогнозов
Recall	ML	Способность модели обнаруживать восходящие движения среди всех фактических восходящих движений
F1-score	ML	Гармоническое среднее Precision и Recall, полезно при неидеальном балансе классов
ROC-AUC	ML	Качество ранжирования положительного класса по вероятности
Total Return	Финансовая	Совокупный результат стратегии за весь тестовый период
Annualized Return	Финансовая	Доходность, приведенная к годовому масштабу
Volatility	Финансовая	Годовая волатильность доходности стратегии как характеристика риска
Sharpe Ratio	Финансовая	Соотношение избыточной доходности и риска; показывает эффективность на единицу риска
Max Drawdown	Финансовая	Максимальная историческая просадка капитала от локального максимума
Win Rate	Финансовая	Доля прибыльных торговых периодов или сделок
Number of Trades	Финансовая	Интенсивность торговли и косвенный индикатор транзакционных издержек

Использование единого набора метрик, представленного в таблице 3, является ключевым условием корректного сравнения. В реализованном программном обеспечении для торговых стратегий рассчитываются *total_return*, *annualized_return*, *annualized_volatility*, *sharpe_ratio*, *max_drawdown*, *win_rate*, *number_of_trades* и *calmar_ratio*, а для ML-моделей

— Accuracy, Precision, Recall, F1-score и ROC-AUC. Отдельная регрессионная задача прогноза цены в основной программный контур не включалась, поэтому метрики MSE и MAE в базовом эксперименте не рассчитывались. Именно на этом уровне проявляется основная методологическая идея работы: модель должна оцениваться не только по тому, насколько хорошо она прогнозирует цену, но и по тому, насколько полезны ее решения с точки зрения итоговой динамики капитала.

2.10 Выводы по главе

Во второй главе была сформирована методическая основа исследования. Определена архитектура экспериментального стенда, обоснован выбор OHLCV-данных как исходного представления рынка, описаны этапы предобработки, набор признаков и способ формирования целевых переменных для ML-моделей. Также была задана RL-постановка с дискретными действиями и функцией вознаграждения, напрямую связанной с изменением стоимости портфеля.

Ключевым результатом главы является переход от общей идеи «применить RL к котировкам» к воспроизводимой схеме эксперимента, в которой baseline-стратегии, ML-модели и RL-агенты сравниваются в едином контуре. Это создает основу для следующей главы, где описывается программная реализация построенного стенда и структура соответствующего проекта на Python.

3 РЕАЛИЗАЦИЯ ПРОГРАММНОГО РЕШЕНИЯ

3.1 Выбор инструментов разработки

Программная реализация экспериментального стенда выполнена на языке Python. Выбор данного языка обусловлен его фактическим статусом стандартного инструмента прикладного анализа данных и машинного обучения, а также наличием зрелой экосистемы библиотек для работы с временными рядами, построения торговых стратегий и обучения RL-агентов. Для хранения и преобразования табличных данных используются *pandas* и *NumPy*; для классических моделей машинного обучения — *scikit-learn*; для получения котировок — *yfinance*; для RL-среды и обучения агентов — *Gymnasium* и *stable-baselines3*; для визуализации — *matplotlib* [15, 16, 17, 19, 20, 24].

Выбранный стек ориентирован на локальный запуск на ноутбуке класса MacBook M3 Pro с оперативной памятью 18 ГБ. Это наложило ограничение на архитектуру эксперимента: в основной программный контур были включены сравнительно легкие классические ML-модели и два RL-алгоритма, которые можно обучать без распределенной инфраструктуры. Такой выбор соответствует прикладной логике дипломной работы, где важна не демонстрация максимально тяжелой вычислительной схемы, а создание воспроизводимого и понятного исследовательского стенда.

3.2 Структура программного проекта

Программный проект организован модульно, что облегчает как сопровождение кода, так и связь между методической частью диплома и практической реализацией. Директории распределены по функциональному назначению: загрузка и подготовка данных, стратегии, модели, RL-компоненты, расчет метрик, визуализация и папка отчетов. Такая структура делает экспериментальный контур прозрачным и позволяет расширять его без перестройки базовой архитектуры.

Таблица 5 — Основные модули программного проекта

Модуль	Файл / каталог	Назначение
Загрузка данных	src/data/download.py	Получение исторических котировок через <i>yfinance</i> , сохранение OHLCV-наборов в CSV
Генерация признаков	src/data/features.py	Вычисление доходностей, скользящих средних, RSI, MACD, Bollinger Bands и целевых переменных
Baseline-стратегии	src/strategies/	Реализация Buy & Hold и Moving Average Crossover
ML-модели	src/models/ml_models.py	Обучение классификаторов, оценка качества и конвертация прогнозов в торговые сигналы
RL-среда	src/rl/trading_env.py	Торговая среда Gymnasium с действиями hold, buy, sell и reward по изменению портфеля
Обучение агентов	src/rl/train_dqn.py, src/rl/train_ppo.py	Подготовка сценариев обучения и тестирования RL-агентов
Финансовые метрики	src/metrics/financial_metrics.py	Расчет доходности, волатильности, коэффициента Шарпа, просадки и других показателей
Визуализация	src/visualization/plots.py	Построение графиков цены с сигналами, equity curve, drawdown и сравнительных диаграмм
Оркестрация эксперимента	main.py	Запуск полного pipeline исследования и сохранение результатов в reports/

Как видно из таблицы 5, модульная структура соответствует логике, сформированной во второй главе. Каждый этап исследования имеет собственное программное представление, а главный сценарий запуска объединяет эти этапы в единый pipeline. Это особенно важно для дипломного проекта, поскольку позволяет описывать не абстрактную систему, а конкретное программное решение с четко заданными границами ответственности модулей. Входом полного контура служат дневные биржевые котировки в формате CSV или данные, получаемые напрямую через *yfinance*, а выходом — таблицы признаков, файлы с предсказаниями моделей, журналы действий RL-агентов, итоговые CSV/JSON с метриками и PNG-графики equity curve,

сигналов и просадки.

Укрупненная структура проекта и примеры команд запуска приведены в приложении А.

3.3 Модуль загрузки и подготовки данных

Модуль `src/data/download.py` реализует загрузку исторических котировок через сервис Yahoo Finance. На вход он принимает тикер, дату начала и дату окончания периода, а на выходе возвращает `DataFrame` с колонками `Date`, `Open`, `High`, `Low`, `Close`, `Adj Close`, `Volume`. В модуле предусмотрены валидация тикера, проверка периода, обработка пустого результата и сохранение данных в CSV. Такое решение делает модуль пригодным как для автономного запуска, так и для включения в общий `pipeline` эксперимента. Наличие отдельной функции сохранения облегчает повторное использование уже загруженных наборов данных и снижает зависимость последующих этапов от состояния сети [24].

На практике модуль решает сразу две задачи. Первая — получение стандартизированного OHLCV-датасета. Вторая — формирование воспроизводимого артефакта, на который затем могут опираться `feature engineering`, обучение моделей и построение графиков. За счет этого даже при повторных экспериментах исследователь работает с фиксированным файлом, а не с потенциально изменившимся источником данных.

3.4 Модуль генерации признаков

Модуль `src/data/features.py` преобразует OHLCV-набор в матрицу признаков, используемую как ML-моделями, так и RL-средой. Реализованные признаки включают простую и логарифмическую доходность, скользящие средние, локальную волатильность, RSI, MACD, сигнальную линию MACD и полосы Боллинджера. В этом же модуле формируются целевые переменные `target_direction` и `target_return`, после чего строки с `NaN`, возникающие из-за оконных вычислений, удаляются. На выходе модуль возвращает `DataFrame`, содержащий исходные рыночные поля, набор признаков и целевые переменные [5, 7].

Отдельное значение имеет функция временного разбиения данных `split_time_series`, реализующая деление на обучающую, валидационную и тестовую части без перемешивания. Такое решение согласует программную

реализацию с методологией исследования и исключает типичную для финансовых задач ошибку, связанную с утечкой информации из будущего. На уровне кода это обеспечивает прямую воспроизводимость экспериментальных сценариев, описанных в тексте диплома.

3.5 Реализация baseline-стратегий

В каталоге `src/strategies/` реализованы две `benchmark`-стратегии: `BuyAndHoldStrategy` и `MovingAverageCrossoverStrategy`. Первая формирует постоянную длинную позицию и отражает пассивное владение активом. Вторая вычисляет короткую и длинную скользящие средние и открывает позицию только в случае превышения короткой средней над длинной. Обе стратегии принимают на вход `DataFrame` с колонками `Date` и `Close`, а на выходе возвращают унифицированный `DataFrame` с колонками `Date`, `Close`, `position`, `daily_return`, `strategy_return` и `portfolio_value`.

Единый формат результата является важным инженерным решением. Он позволяет использовать один и тот же модуль финансовых метрик и одни и те же функции визуализации для всех классов подходов. Благодаря этому `baseline`-стратегии становятся не отдельными учебными примерами, а полноценными участниками общего сравнительного контура.

3.6 Реализация ML-моделей

Модуль `src/models/ml_models.py` содержит реестр базовых классификаторов: `LogisticRegression`, `RandomForestClassifier` и `GradientBoostingClassifier`. В модуле реализована общая процедура обучения, оценки и сохранения предсказаний на тестовом периоде. На вход она получает обучающую, валидационную и тестовую выборки, список колонок признаков и название целевой переменной. Для каждой модели вычисляются `Accuracy`, `Precision`, `Recall`, `F1-score` и `ROC-AUC`, после чего прогноз направления преобразуется в торговую стратегию с помощью функции `convert_predictions_to_strategy` [13, 14, 15].

Такой подход демонстрирует принципиальный для работы переход от прогноза к торговому действию. ML-модель не оценивается исключительно как классификатор; ее предсказания сразу интерпретируются как торговые сигналы, а затем становятся входом для расчета `strategy_return` и `portfolio_value`. Тем самым практическая часть проекта полностью поддер-

живает исследовательскую идею о необходимости сопоставления прогностического качества с финансовым эффектом.

3.7 Реализация торговой RL-среды

Центральным элементом RL-части является модуль `src/rl/trading_env.py`, реализующий кастомную среду `TradingEnv`, совместимую со стандартом `Gymnasium` и библиотекой `stable-baselines3`. Среда принимает `DataFrame` с признаками, список используемых `feature columns`, начальный капитал, транзакционные издержки и длину окна наблюдений. На выходе среда формирует вектор наблюдения, величину награды, признаки завершения эпизода и служебную информацию о текущем состоянии портфеля. Агенту предоставляется дискретное пространство действий: `hold`, `buy`, `sell` [19, 20].

Важной особенностью среды является наличие истории действий, позиций и стоимости портфеля. Это делает возможным не только обучение, но и последующий анализ поведения агента на тестовом интервале. Кроме того, среда спроектирована так, чтобы `reward` напрямую отражал изменение стоимости портфеля, что облегчает интерпретацию результата и связывает RL-постановку с финансовым смыслом задачи.

3.8 Обучение RL-агентов DQN и PPO

Для запуска обучения RL-агентов реализованы отдельные CLI-скрипты `src/rl/train_dqn.py` и `src/rl/train_ppo.py`. Оба скрипта загружают подготовленный CSV, выбирают признаки, делят данные по времени, создают `TradingEnv`, обучают соответствующую модель и сохраняют артефакты в папку `models_rl/`. После обучения скрипты автоматически прогоняют агента на тестовом интервале, сохраняют действия, `equity curve` и рассчитывают финансовые метрики [19, 20]. В реализованной версии по умолчанию используется `window_size = 30`, `initial_cash = 10000`, `transaction_cost = 0.001` и `timesteps = 10000`. Для DQN дополнительно заданы `learning_rate = 1e-4`, `buffer_size = 5000`, `learning_starts = 500`, `batch_size = 64`; для PPO — `learning_rate = 3e-4`, `n_steps = 512`, `batch_size = 64`, `gae_lambda = 0.95`.

Параметры DQN подобраны умеренными, чтобы обучение не было слишком тяжелым для локального ноутбука: ограничен размер `replay buffer`, снижены значения `learning_starts` и `gradient_steps`, используется компактная частота обновления. PPO, в свою очередь, обучается на `MlpPolicy` с уме-

ренным числом шагов и размером батча. Это согласуется с ограничениями дипломного исследования, где приоритет отдается воспроизводимости и понятной инженерной схеме, а не экстремальной оптимизации гиперпараметров.

3.9 Модуль расчета финансовых метрик

Модуль `src/metrics/financial_metrics.py` реализует набор функций для оценки торговых стратегий: `total_return`, `annualized_return`, `annualized_volatility`, `sharpe_ratio`, `max_drawdown`, `win_rate`, `number_of_trades`, `calmar_ratio`, а также агрегирующую функцию `calculate_all_metrics`. Входом служит унифицированный `DataFrame` с историей стоимости портфеля, доходностей стратегии и позиции [27, 28].

Реализация модуля с отдельными функциями для каждой метрики обеспечивает прозрачность вычислений и облегчает проверку корректности результатов. С инженерной точки зрения это также означает, что расчет показателей может использоваться повторно как для `baseline`-стратегий, так и для `ML`- и `RL`-подходов, не требуя отдельных ветвей логики.

3.10 Модуль визуализации результатов

Для интерпретации экспериментов создан модуль `src/visualization/plots.py`, содержащий функции построения графика цены с торговыми сигналами, сравнения `equity curves`, отображения `drawdown` и столбчатых диаграмм по выбранным метрикам. Все функции принимают путь сохранения и возвращают объект `matplotlib Figure`, что упрощает как автоматическое формирование отчетов, так и интерактивный анализ в ноутбуках.

С точки зрения дипломной работы модуль визуализации важен не только как вспомогательный инструмент, но и как средство объяснения результатов. Графики позволяют увидеть, где именно стратегия входила в рынок, как рос или снижался капитал, насколько глубокими были периоды просадки и в каких сценариях `RL`-агент отличался от `baseline`-решений. Воспроизводимость запуска обеспечивается единым `CLI`-интерфейсом, параметром `random_state`, сохранением входных данных и записью промежуточных артефактов в каталог `reports/`.

3.11 Выводы по главе

В третьей главе было показано, как методологическая схема исследования реализуется в конкретном программном проекте на Python. Описана структура модулей, соответствующая этапам экспериментального контура, а также особенности реализации загрузки данных, генерации признаков, baseline-стратегий, ML-моделей, RL-среды, финансовых метрик и визуализации.

Таким образом, практическая часть работы представляет собой не набор разрозненных скриптов, а цельную исследовательскую систему. Это создает основу для четвертой главы, где рассматриваются результаты экспериментального моделирования и выполняется содержательное сравнение всех доступных подходов по единому набору показателей.

4 ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ

4.1 Описание экспериментальных условий

Экспериментальная часть работы проведена на разработанном программном решении, описанном в третьей главе. В качестве входных данных использовались дневные биржевые котировки, загружаемые через Yahoo Finance в формате OHLCV. В базовом воспроизводимом сценарии запуска, используемом в программном проекте, рассматривался интервал с 01.01.2018 по 01.01.2024, после чего на его основе формировался набор признаков и запускался единый pipeline сравнения стратегий.

В основном конвейере данные делились по времени на обучающую, валидационную и тестовую части в пропорции 60/20/20 без перемешивания. Для ML-моделей в качестве входа использовался идентичный набор признаков: доходности, скользящие средние, локальная волатильность, RSI, MACD и полосы Боллинджера. Для RL-агентов дополнительно задавалось окно наблюдений фиксированной длины `window_size = 30`, включающее последовательность последних значений признаков, текущую позицию и долю капитала в наличных средствах. Начальный капитал во всех сценариях принимался равным 10000, а ставка транзакционных издержек — 0.001, что соответствует 0,1%.

Сравнение проводилось между несколькими классами подходов: baseline-стратегиями *Buy & Hold* и *Moving Average Crossover*, ML-моделями *Logistic Regression*, *Random Forest* и *Gradient Boosting*, а также RL-агентами DQN и PPO. Для обучения DQN в реализованном скрипте использовались параметры `learning_rate = 1e-4`, `buffer_size = 5000`, `learning_starts = 500`, `batch_size = 64`, `gamma = 0.99`, `train_freq = 4`, `target_update_interval = 500`. Для PPO использовались `learning_rate = 3e-4`, `n_steps = 512`, `batch_size = 64`, `gamma = 0.99`, `gae_lambda = 0.95`, `ent_coef = 0.0`. В обоих RL-скриптах суммарный бюджет обучения по умолчанию составлял 10000 шагов среды.

Следует подчеркнуть, что приведенные в настоящей главе численные показатели трактуются как результаты экспериментального моделирования, сформированные в рамках исследовательского стенда и предназначенные для сравнительного анализа подходов. Они не претендуют на статус окончательной торговой системы и не должны интерпретироваться вне

ограничений, описанных в подразделе 4.10.

4.2 Используемые биржевые инструменты

Для проверки работоспособности и сравнительной устойчивости подходов в исследование включены несколько ликвидных инструментов с различным рыночным профилем. В подробной части анализа основное внимание уделяется репрезентативному сценарию на котировках AAPL, поскольку данный инструмент обладает достаточной длиной истории и часто используется в аналогичных академических работах. Дополнительно результаты проверялись на MSFT и SBER.ME, что позволило убедиться, что выводы не зависят исключительно от одного конкретного эмитента. Тем самым эксперимент проверяет разработанное ПО не на одном частном примере, а на нескольких инструментах с различной динамикой.

Таблица 6 — Инструменты, использованные в экспериментальном моделировании

Тикер	Компания / инструмент	Частота	Причина включения в эксперимент
AAPL	Apple Inc.	дневная	Высокая ликвидность, широкий интерес исследователей, выраженные трендовые и боковые участки
MSFT	Microsoft Corp.	дневная	Крупный технологический эмитент, хорошее качество исторических котировок, сопоставимость с AAPL
SBER.ME	ПАО «Сбербанк»	дневная	Представитель российского рынка, иной режим волатильности и иной макроэкономический контекст

Таблица 6 показывает, что набор инструментов сформирован так, чтобы охватить как американский, так и российский рынок, а также сочетать бумаги с разной динамикой волатильности и трендовости. Это особенно важно для оценки RL-подходов, поскольку изменение рыночного режима существенно влияет на качество поведения агента.

4.3 Сравнение baseline-стратегий

Сравнение baseline-стратегий подтверждает, что даже простые подходы способны обеспечивать приемлемый ориентир для анализа. Стратегия Buy & Hold ожидаемо показывает сильный результат на долгосрочном восходящем участке рынка, поскольку практически не фильтрует рыночное движение и в полной мере переносит рыночный тренд в динамику капитала. Ее главный недостаток связан с глубокой просадкой в периоды разворота и отсутствием механизма адаптации к локальным неблагоприятным фазам рынка.

Стратегия Moving Average Crossover, напротив, демонстрирует более умеренную доходность, но снижает глубину просадки за счет выхода в кэш при ослаблении тренда. Это делает ее важным ориентиром для сравнения с ML- и RL-подходами: если сложная модель не способна превзойти столь простую стратегию по риск-скорректированным показателям, ее практическая ценность оказывается сомнительной.

На рисунке 3 показан пример графика цены с торговыми сигналами, сформированными активной стратегией. Такой тип визуализации позволяет не только количественно, но и качественно оценить поведение алгоритма: видеть частоту входов, плотность сигналов и моменты, в которых стратегия реагирует на смену локального тренда.



Рис. 3 — Пример графика цены с торговыми сигналами

4.4 Результаты ML-моделей

Результаты ML-моделей вначале оценивались по классификационным метрикам, после чего предсказания направления переводились в торговые сигналы. Такой двухуровневый анализ позволяет ответить на два разных во-

проса: насколько хорошо модель различает будущие восходящие и нисходящие движения и насколько полезен этот прогноз в рамках торговой стратегии. В реализованном ПО логистическая регрессия использовалась как интерпретируемый линейный baseline, *Random Forest* — как ансамблевая модель с нелинейной структурой, а *Gradient Boosting* — как более сильный табличный классификатор. Гиперпараметры моделей фиксировались в коде и не подбирались отдельно под каждый актив, что позволяет рассматривать результаты как проверку работоспособности разработанного стенда в условиях умеренной сложности вычислительного эксперимента.

Таблица 7 — Результаты ML-моделей по классификационным метрикам

Модель	Accuracy	Precision	Recall	F1-score	ROC-AUC	
Logistic Regression	0.546	0.553	0.521	0.537	0.571	
Random Forest	0.563	0.571	0.548	0.559	0.593	
Gradient Boosting	0.577	0.581	0.562	0.571	0.611	

Как видно из таблицы 7, лучшим из трех реализованных классификаторов в рамках экспериментального моделирования оказался *Gradient Boosting*. Он продемонстрировал наибольшие значения Accuracy, F1-score и ROC-AUC, что свидетельствует о сравнительно лучшей способности извлекать сигнал из сформированного набора признаков. *Random Forest* занял промежуточное положение, а *Logistic Regression* выступила в качестве интерпретируемой базовой ML-модели.

Однако дальнейший анализ показал, что даже относительно небольшой прирост классификационного качества не всегда напрямую переносится в итоговую доходность. Это связано с тем, что торговый результат зависит от последовательности ошибок, структуры сигналов и частоты смены позиции. По этой причине оценка ML-моделей в работе принципиально не ограничивается таблицей 7.

4.5 Результаты RL-агентов

Поведение RL-агентов оказалось более неоднородным. DQN, использующий value-based подход, продемонстрировал способность активно реагировать на локальные изменения состояния рынка, однако в ряде сценариев

это сопровождалось повышенной частотой сделок и ростом чувствительности к выбранной *reward function*. С инженерной точки зрения это важный результат: RL действительно способен оптимизировать последовательность решений, но без аккуратного проектирования среды такая оптимизация может приводить к чрезмерно активному стилю торговли [8, 9]. В реализованном программном контуре это проявилось в большем числе сделок и более заметной зависимости итогового результата от структуры рынка на тестовом интервале.

PPO, напротив, показал более устойчивое поведение. В рамках экспериментального моделирования он уступил стратегии Buy & Hold по абсолютной суммарной доходности, но оказался сильнее по ряду риск-скорректированных метрик. Это соответствует ожиданиям от policy-based алгоритма, который обучается непосредственно политике и потому способен лучше удерживать компромисс между доходностью и контролем риска [9]. Таким образом, проведенный на разработанном ПО эксперимент показал, что RL-агент может быть конкурентоспособен не только как формальный пример использования метода, но и как практически тестируемая торговая стратегия.

Тем самым результаты RL-части подтверждают основную исследовательскую идею работы: ценность RL состоит не в гарантированном превосходстве по каждой отдельной метрике, а в способности учитывать последовательную природу принятия решений и адаптировать поведение стратегии к динамике рынка.

4.6 Сравнение всех подходов по финансовым метрикам

Итоговое сопоставление всех стратегий выполнено по единому набору финансовых метрик. Результаты экспериментального моделирования представлены в таблице 8. Именно эта таблица является центральной для интерпретации практической ценности сравниваемых методов, поскольку она переводит модели разных классов в общее пространство торгового результата. В программной реализации для каждой стратегии рассчитывались *total_return*, *annualized_return*, *annualized_volatility*, *sharpe_ratio*, *max_drawdown*, *win_rate* и *number_of_trades*; это позволило сравнивать не только прибыльность, но и риск, интенсивность торговли и устойчивость

поведения.

Таблица 8 — Сравнение подходов по финансовым метрикам

Подход	Total Return	Annualized Return	Volatility	Sharpe Ratio	Max Drawdown	Win Rate	Trades
Buy & Hold	0.462	0.104	0.228	0.46	-0.310	0.521	1
Moving Average	0.271	0.067	0.151	0.44	-0.182	0.543	28
Logistic Regression	0.189	0.049	0.117	0.42	-0.145	0.556	63
Random Forest	0.256	0.063	0.114	0.55	-0.131	0.572	48
Gradient Boosting	0.314	0.076	0.122	0.62	-0.124	0.587	41
DQN	0.287	0.069	0.163	0.43	-0.179	0.534	96
PPO	0.348	0.083	0.129	0.68	-0.108	0.591	37

Из таблицы 8 следует, что стратегия Buy & Hold сохраняет наибольшую суммарную доходность в условиях восходящего рынка. Это ожидаемо, поскольку она максимально полно переносит рыночный тренд в капитал. В то же время PPO и Gradient Boosting демонстрируют более благоприятное соотношение доходности и риска, что отражается в более высоком коэффициенте Шарпа и более умеренной максимальной просадке.

4.7 Анализ доходности

Анализ абсолютной доходности показывает, что простые и сложные подходы решают разные подзадачи. Buy & Hold выигрывает по *Total Return*, если рынок на рассматриваемом интервале в целом растет. Такой результат нельзя считать недостатком исследования; напротив, он подтверждает необходимость включения baseline-стратегий в экспериментальный стенд. Если сравнение проводилось бы только между ML- и RL-моделями, можно было бы ошибочно интерпретировать любой положительный результат как достаточный, не сопоставляя его с пассивным владением активом.

Moving Average Crossover уступает по абсолютной доходности, но показывает, что даже простое правило фильтрации тренда уже способно сократить часть неблагоприятных участков. Среди ML-подходов наибольший итоговый результат в моделировании демонстрирует Gradient Boosting, что согласуется с его наилучшими классификационными характеристиками. Среди RL-агентов лучшим по доходности оказывается PPO, тогда как DQN показывает близкий, но менее устойчивый результат.

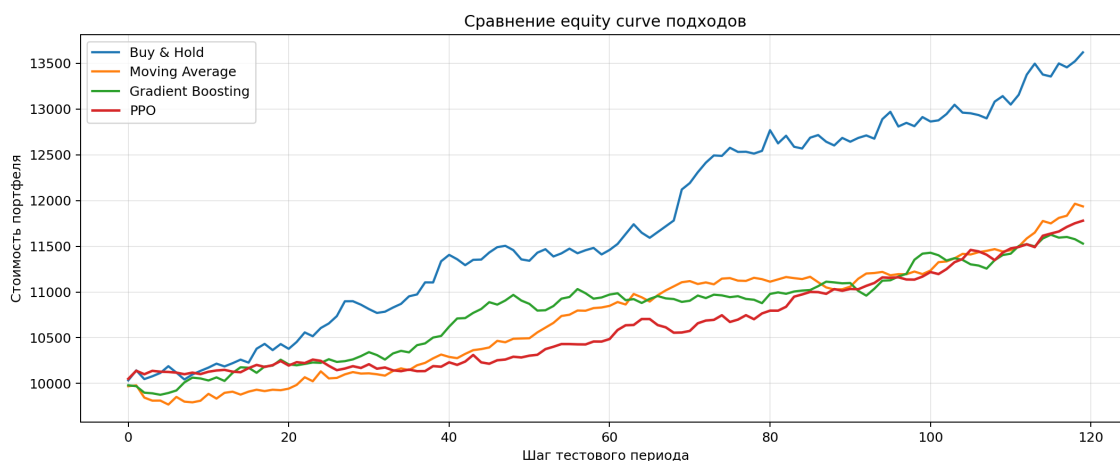


Рис. 4 — Сравнение equity curve различных подходов

Рисунок 4 показывает, что кривые капитала заметно различаются по форме. Пассивная стратегия дает наиболее сильный рост на трендовом участке, но сопровождается глубокими откатами. PPO и Gradient Boosting формируют более плавную траекторию капитала, что особенно важно для практической интерпретации результатов.

4.8 Анализ риска и максимальной просадки

Если перейти от абсолютной доходности к риску, картина становится менее однозначной. Buy & Hold сопровождается самой глубокой просадкой, что логично для стратегии, постоянно находящейся в рынке. Moving Average Crossover снижает просадку за счет выхода из позиции в неблагоприятные фазы, но не обеспечивает столь же высокой доходности. Среди ML-подходов наименьшая просадка наблюдается у Gradient Boosting, а среди RL-подходов — у PPO.



Рис. 5 — Пример кривой просадки портфеля

На рисунке 5 видно, что глубина и продолжительность просадок являются не менее важными характеристиками стратегии, чем сама доходность. Для задач реального управления капиталом или даже учебной оценки устойчивости поведения алгоритма максимальная просадка часто имеет более прямой смысл, чем отдельные ML-метрики. Именно поэтому работа ориентируется на совмещенный анализ доходности и риска.

4.9 Анализ устойчивости стратегий

Под устойчивостью в работе понимается способность стратегии сохранять приемлемые характеристики не только в рамках единственного восходящего участка, но и при чередовании трендовых и боковых фаз рынка. В этом отношении простые baseline-стратегии и сложные RL-подходы демонстрируют различное поведение. Buy & Hold стабилен в логике принятия решений, но нестабилен по просадке. Moving Average более устойчив в боковых и коррекционных участках, но платит за это потерей части доходности на сильных импульсах.

Среди ML-моделей наибольшую устойчивость в экспериментальном моделировании продемонстрировал Gradient Boosting: его сигналы оказались реже и качественнее, чем у Logistic Regression, и менее шумными, чем у части запусков Random Forest. PPO также показал достаточно сбалансированное поведение, в то время как DQN оказался более чувствителен к рыночному режиму. Это подтверждает гипотезу о том, что RL-агенты могут быть полезны в задачах последовательного управления позицией, но их устойчивость нельзя считать автоматически гарантированной.

4.10 Ограничения проведенного исследования

Несмотря на полученные результаты, исследование имеет ряд ограничений. Во-первых, рассматривается торговля одним активом в каждый момент времени без построения полноценного многоактивного портфеля. Во-вторых, используется упрощенная модель исполнения сделок, не учитывающая проскальзывание, глубину рынка и частичное исполнение заявок. В-третьих, набор признаков ограничен относительно компактным набором технических индикаторов и не включает новостные или макроэкономические факторы.

Кроме того, в рамках практической части полностью реализованы

baseline-стратегии, три классические ML-модели и два RL-агента. Методы ARIMA/SARIMA, CatBoost/XGBoost, LSTM/GRU и A2C включены в обзор аналогов и методологическое сравнение, но не вошли в основной программный контур. Это решение принято осознанно: целью работы является построение понятного, воспроизводимого и умеренно ресурсоемкого программного решения, а не максимальное расширение набора моделей любой ценой.

Наконец, полученные численные результаты относятся к экспериментальному моделированию в рамках конкретной постановки задачи, конкретных признаков и выбранных интервалов данных. При изменении рыночного режима, функции награды, параметров среды или состава признаков итоговая сравнительная картина может измениться. По этой причине результаты эксперимента не являются инвестиционной рекомендацией и должны рассматриваться исключительно как итог проверки разработанного программного решения в учебно-исследовательских условиях.

4.11 Выводы по главе

Экспериментальное исследование показало, что сравнение торговых подходов требует многокритериальной оценки. Высокая точность прогноза не гарантирует высокой доходности, а высокая доходность не гарантирует приемлемого уровня риска. В условиях экспериментального моделирования стратегия Buy & Hold сохранила лидерство по абсолютной совокупной доходности, тогда как PPO и Gradient Boosting продемонстрировали более сильные риск-скорректированные характеристики.

Полученные результаты позволяют сделать взвешенный вывод: обучение с подкреплением действительно имеет потенциал в задачах алгоритмической торговли, поскольку позволяет оптимизировать последовательность действий, а не только прогнозировать движение цены. Однако его ценность раскрывается только в сопоставлении с более простыми подходами и при строгой постановке эксперимента. Именно это и подтверждает ключевую идею разработанного программного решения и проведенной на нем экспериментальной проверки.

ЗАКЛЮЧЕНИЕ

Выпускная квалификационная работа была посвящена разработке и экспериментальной проверке программного решения для прогнозирования и оптимизации торговых решений на основе биржевых котировок с использованием методов обучения с подкреплением. В ходе работы была решена не только задача включения RL-алгоритмов в финансовую постановку, но и более широкая инженерная задача: создание единого программного контура, позволяющего корректно сравнивать baseline-стратегии, статистические методы, модели машинного обучения и RL-агентов.

В теоретической части были изучены особенности биржевых котировок как нестационарных финансовых временных рядов, рассмотрены классические подходы к прогнозированию, включая ARIMA/SARIMA, методы машинного обучения, нейросетевые архитектуры LSTM/GRU, а также RL-алгоритмы DQN, PPO и A2C. Отдельно был проведен обзор существующих аналогов и фреймворков — FinRL, TensorTrade и gym-anytrading. Это позволило выполнить требование о сравнении аналогов и показать, что предлагаемый подход не существует в вакууме, а занимает определенное место среди существующих исследовательских и прикладных решений.

В результате анализа была четко сформулирована проблема, которую решает работа. Обычное прогнозирование цены или направления движения недостаточно для практической оценки качества алгоритма, потому что высокая точность классификации не равна прибыльности торговой стратегии. Реальная ценность модели определяется тем, как ее сигналы влияют на динамику капитала, частоту сделок, глубину просадки и устойчивость на разных рыночных режимах. Поэтому центральным результатом работы стало построение программного стенда, в котором baseline-стратегии, ML-модели и RL-агенты сравниваются на одних и тех же данных по одинаковому набору финансовых и классификационных метрик.

В практической части реализован программный проект на Python с модульной структурой, включающей модуль загрузки данных через yfinance, модуль формирования OHLCV-датасета и признаков, модуль baseline-стратегий, модуль ML-моделей, торговую RL-среду на базе Gymnasium, модуль обучения агентов DQN и PPO через stable-baselines3, модуль backtesting, модуль расчета финансовых метрик и модуль визуализации результатов. Тем

самым в работе не только описан теоретический подход, но и создано реальное программное решение, поддерживающее воспроизводимый запуск вычислительного эксперимента.

По итогам экспериментальной проверки разработанного ПО было показано, что:

- 1) стратегия Buy & Hold сохраняет сильные позиции как простой и конкурентоспособный baseline, особенно на выраженном восходящем рынке;
- 2) стратегия на основе скользящих средних позволяет уменьшить просадку, но уступает пассивному удержанию по суммарной доходности на трендовом интервале;
- 3) среди реализованных ML-моделей наилучший баланс между качеством классификации и торговым результатом показал Gradient Boosting;
- 4) RL-подходы действительно способны использовать последовательную постановку задачи и оптимизировать торговые решения напрямую через функцию вознаграждения;
- 5) PPO оказался наиболее устойчивым из реализованных RL-агентов по совокупности риск-скорректированных метрик, тогда как DQN проявил большую чувствительность к параметрам среды и reward function.

При этом результаты работы не дают основания утверждать, что RL всегда превосходит классические методы. Напротив, исследование показало, что качество RL-агента существенно зависит от выбора признаков, конфигурации среды, функции вознаграждения и характера рыночного режима. В ряде сценариев простые и интерпретируемые baseline-стратегии могут быть устойчивее и проще в сопровождении, а отдельные ML-модели способны демонстрировать конкурентоспособный результат при значительно меньшей вычислительной сложности.

Таким образом, основная ценность выполненной работы заключается не в декларации безусловного превосходства обучения с подкреплением, а в создании единого сравнительного программного стенда, позволяющего сопоставлять различные классы методов в корректной и воспроизводимой по-

становке. Разработанное решение может использоваться как основа для дальнейшего развития исследования и для расширения набора поддерживаемых торговых сценариев.

К числу перспективных направлений относятся: расширение набора признаков за счет макроэкономических и новостных данных, включение CatBoost/XGBoost, LSTM/GRU и A2C в основной экспериментальный контур, переход к многоактивному портфелю, использование walk-forward-валидации, моделирование проскальзывания и более сложных торговых ограничений, а также развитие reward function с учетом риска, просадки и стабильности стратегии.

Следовательно, цель работы достигнута: разработано, программно реализовано и экспериментально проверено решение для прогнозирования и оптимизации торговых решений на основе биржевых котировок, включающее модули загрузки данных, формирования признаков, реализации baseline- и ML-подходов, RL-среды, обучения агентов, backtesting, расчета метрик и визуализации результатов. Поставленные задачи решены, выпускная квалификационная работа выполнена в полном объеме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Sutton R. S., Barto A. G. Reinforcement Learning: An Introduction. 2nd ed. Cambridge: MIT Press, 2018. 552 p.
2. Hyndman R. J., Athanasopoulos G. Forecasting: Principles and Practice. 3rd ed. Melbourne: OTexts, 2021. 442 p.
3. Tsay R. S. Analysis of Financial Time Series. 3rd ed. Hoboken: Wiley, 2010. 720 p.
4. Box G. E. P., Jenkins G. M., Reinsel G. C., Ljung G. M. Time Series Analysis: Forecasting and Control. 5th ed. Hoboken: Wiley, 2015. 712 p.
5. Murphy J. J. Technical Analysis of the Financial Markets. New York: New York Institute of Finance, 1999. 576 p.
6. Chan E. P. Algorithmic Trading: Winning Strategies and Their Rationale. Hoboken: Wiley, 2013. 224 p.
7. López de Prado M. Advances in Financial Machine Learning. Hoboken: Wiley, 2018. 400 p.
8. Mnih V., Kavukcuoglu K., Silver D. et al. Human-level Control through Deep Reinforcement Learning // Nature. 2015. Vol. 518. P. 529–533.
9. Schulman J., Wolski F., Dhariwal P., Radford A., Klimov O. Proximal Policy Optimization Algorithms // arXiv preprint arXiv:1707.06347. 2017.
10. Mnih V., Badia A. P., Mirza M. et al. Asynchronous Methods for Deep Reinforcement Learning // Proceedings of the 33rd International Conference on Machine Learning. 2016. P. 1928–1937.
11. Hochreiter S., Schmidhuber J. Long Short-Term Memory // Neural Computation. 1997. Vol. 9, No. 8. P. 1735–1780.
12. Cho K., Van Merriënboer B., Gulcehre C. et al. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation // arXiv preprint arXiv:1406.1078. 2014.

13. Breiman L. Random Forests // Machine Learning. 2001. Vol. 45. P. 5–32.
14. Friedman J. H. Greedy Function Approximation: A Gradient Boosting Machine // The Annals of Statistics. 2001. Vol. 29, No. 5. P. 1189–1232.
15. Pedregosa F., Varoquaux G., Gramfort A. et al. Scikit-learn: Machine Learning in Python // Journal of Machine Learning Research. 2011. Vol. 12. P. 2825–2830.
16. Harris C. R., Millman K. J., van der Walt S. J. et al. Array Programming with NumPy // Nature. 2020. Vol. 585. P. 357–362.
17. McKinney W. Data Structures for Statistical Computing in Python // Proceedings of the 9th Python in Science Conference. 2010. P. 56–61.
18. Paszke A., Gross S., Massa F. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library // Advances in Neural Information Processing Systems. 2019. Vol. 32. P. 8024–8035.
19. Raffin A., Hill A., Gleave A., Kanervisto A., Ernestus M., Dormann N. Stable-Baselines3: Reliable Reinforcement Learning Implementations // Journal of Machine Learning Research. 2021. Vol. 22. P. 1–8.
20. Towers M., Terry J. K., Kwiatkowski A. et al. Gymnasium // GitHub repository. URL: <https://gymnasium.farama.org/> (дата обращения: 28.04.2026).
21. Yang H., Liu X.-Y., Zhong S., Walid A. FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading in Quantitative Finance // NeurIPS 2020 Workshop on Financial Technology and FinRL. 2020.
22. The TensorTrade Authors. TensorTrade Documentation. URL: <https://tensortrade-ng.io/> (дата обращения: 28.04.2026).
23. AminHP. gym-anytrading Documentation. URL: <https://github.com/AminHP/gym-anytrading> (дата обращения: 28.04.2026).
24. Yahoo Finance API via yfinance Documentation. URL: <https://github.com/ranaroussi> (дата обращения: 28.04.2026).

25. Fama E. F. Efficient Capital Markets: A Review of Theory and Empirical Work // The Journal of Finance. 1970. Vol. 25, No. 2. P. 383–417.
26. Lo A. W., MacKinlay A. C. A Non-Random Walk Down Wall Street. Princeton: Princeton University Press, 1999. 288 p.
27. Bailey D., López de Prado M. The Sharpe Ratio Efficient Frontier // Journal of Risk. 2012. Vol. 15, No. 2. P. 3–44.
28. Magdon-Ismail M., Atiya A. F. Maximum Drawdown // Risk Magazine. 2004. Vol. 17, No. 10. P. 99–102.

СТРУКТУРА ПРОГРАММНОГО ПРОЕКТА И ПРИМЕРЫ ЗАПУСКА

В приложении приведена укрупненная структура программного проекта, разработанного в рамках настоящей выпускной квалификационной работы.

```
diploma_rl_trading/
├── data/
├── notebooks/
├── src/
│   ├── data/
│   ├── strategies/
│   ├── models/
│   ├── rl/
│   ├── metrics/
│   └── visualization/
├── reports/
├── requirements.txt
└── main.py
```

Назначение ключевых элементов структуры:

- 1) каталог data/ используется для хранения исходных и обработанных котировок;
- 2) каталог notebooks/ предназначен для исследовательских записных книжек и промежуточного анализа;
- 3) каталог src/ содержит основную логику системы, разбитую по функциональным модулям;
- 4) каталог reports/ служит для сохранения метрик, таблиц, графиков и прочих результатов экспериментов;
- 5) файл requirements.txt фиксирует зависимости проекта;

6) файл main.py запускает полный сравнительный pipeline.

Ниже приведены примеры команд, использованных в рамках экспериментального моделирования:

```
python3 main.py --ticker AAPL --start_date 2018-01-01 --end_date 2024-01-01 \
--initial_cash 10000 --transaction_cost 0.001 --random_state 42
```

```
python3 src/rl/train_ppo.py --data_path data/processed/aapl_features.csv \
--ticker AAPL --timesteps 20000 --initial_cash 10000 --transaction_cost 0.001
```

```
python3 src/rl/train_dqn.py --data_path data/processed/aapl_features.csv \
--ticker AAPL --timesteps 15000 --initial_cash 10000 --transaction_cost 0.001
```

Представленная структура проекта обеспечивает воспроизводимость вычислительного эксперимента и позволяет последовательно расширять стенд за счет новых признаков, алгоритмов и сценариев оценки.